ELO320 - Estructuras de Datos y Algoritmos
Análisis de Complejidad: F. Recurrencia
Merge Sort

Dr. Nicolás Gálvez R.
nicolas.galvez@usm.cl

Ing. Civil Telemática - Departamento de Electrónica

1er Semestre 2025

UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA

Tiempo de ejecución de un Algoritmo

Definiremos a $T(n)$ como la función de tiempo de ejecución de un algoritmo con input n, y la expresaremos en función a la cantidad de instrucciones. Costo de las operaciones en unidades de tiempo [ut]:

- Instrucciones básicas: 1
- Secuencia de n instrucciones: $c \times n = n$ (c: cte. de costo)
- if-else: $T(\text{condición}) + \max\{T(\text{if}), T(\text{else})\}$
- while: $(n+1) \times T(\text{condición}) + n \times T(\text{while})$

Tiempo de ejecución de un Algoritmo

¿Cuál es la complejidad del siguiente algoritmo?

```
1  int suma(int n) {
2          int sum = 0, i, j;
3          for (i=0; i < n; i++)
4                  for (j=0; j < n; j++)
5                          sum++;
6          return sum;
7  }
```

- Orden del número de iteraciones: n^2.
  - Costo L.2: 3.
  - Costo cada for (L.3, L.4): $1 + (n + 1) + n = 2n + 2$.
  - Costo L.5: 1.
  - Costo L.6: 1.
- Costo for interno (L.4 + L.5): $2n + 2 + n = 3n + 2$.
- Costo de doble for (L.3 + L.4 + L.5):
  $(2n + 2) + n \times (3n + 2) = 3n^2 + 4n + 2$.
- Costo total: $3n^2 + 4n + 6$.

Tiempo de ejecución de un Algoritmo

¿Cuál es la complejidad del siguiente algoritmo?

```
1  int suma(int n) {
2          int sum = 0, i, j;
3          for (i=0; i < n; i++)
4                  for (j=0; j < n; j++)
5                          sum++;
6          return sum;
7  }
```

- Orden del número de iteraciones: n^2.
  - Costo L.2: 3.
  - Costo cada for (L.3, L.4): $1 + (n + 1) + n = 2n + 2$.
  - Costo L.5: 1.
  - Costo L.6: 1.
- Costo for interno (L.4 + L.5): $2n + 2 + n = 3n + 2$.
- Costo de doble for (L.3 + L.4 + L.5): $(2n + 2) + n \times (3n + 2) = 3n^2 + 4n + 2$.
- Costo total: $3n^2 + 4n + 6$.

Tiempo de ejecución de un Algoritmo

¿Cuál es la complejidad del siguiente algoritmo?

```
1  int suma(int n) {
2          int sum = 0, i, j;
3          for (i=0; i < n; i++)
4                  for (j=0; j < n; j++)
5                          sum++;
6          return sum;
7  }
```

- Orden del número de iteraciones: n^2.
  - Costo L.2: 3.
  - Costo cada for (L.3, L.4): $1 + (n + 1) + n = 2n + 2$.
  - Costo L.5: 1.
  - Costo L.6: 1.
- Costo for interno (L.4 + L.5): $2n + 2 + n = 3n + 2$.
- Costo de doble for (L.3 + L.4 + L.5): $(2n + 2) + n \times (3n + 2) = 3n^2 + 4n + 2$.
- Costo total: $3n^2 + 4n + 6$.

Tiempo de ejecución de un Algoritmo

¿Cuál es la complejidad del siguiente algoritmo?

```
1  int suma(int n) {
2          int sum = 0, i, j;
3          for (i=0; i < n; i++)
4                  for (j=0; j < n; j++)
5                          sum++;
6          return sum;
7  }
```

- Orden del número de iteraciones: n^2.
  - Costo L.2: 3.
  - Costo cada for (L.3, L.4): $1 + (n + 1) + n = 2n + 2$.
  - Costo L.5: 1.
  - Costo L.6: 1.
- Costo for interno (L.4 + L.5): $2n + 2 + n = 3n + 2$.
- Costo de doble for (L.3 + L.4 + L.5): $(2n + 2) + n \times (3n + 2) = 3n^2 + 4n + 2$.
- Costo total: $3n^2 + 4n + 6$.

Tiempo de ejecución de un Algoritmo

¿Cuál es la complejidad del siguiente algoritmo?

```
1  int suma(int n) {
2          int sum = 0, i, j;
3          for (i=0; i < n; i++)
4                  for (j=0; j < n; j++)
5                          sum++;
6          return sum;
7  }
```

- Orden del número de iteraciones: n^2.
  - Costo L.2: 3.
  - Costo cada for (L.3, L.4): $1 + (n + 1) + n = 2n + 2$.
  - Costo L.5: 1.
  - Costo L.6: 1.
- Costo for interno (L.4 + L.5): $2n + 2 + n = 3n + 2$.
- Costo de doble for (L.3 + L.4 + L.5): $(2n + 2) + n \times (3n + 2) = 3n^2 + 4n + 2$.
- Costo total: $3n^2 + 4n + 6$.

Tiempo de ejecución de un Algoritmo

¿Cuál es la complejidad del siguiente algoritmo?

```
1  int suma(int n) {
2          int sum = 0, i, j;
3          for (i=0; i < n; i++)
4                  for (j=0; j < n; j++)
5                          sum++;
6          return sum;
7  }
```

- Orden del número de iteraciones: n^2.
  - Costo L.2: 3.
  - Costo cada for (L.3, L.4): $1 + (n + 1) + n = 2n + 2$.
  - Costo L.5: 1.
  - Costo L.6: 1.
- Costo for interno (L.4 + L.5): $2n + 2 + n = 3n + 2$.
- Costo de doble for (L.3 + L.4 + L.5): $(2n + 2) + n \times (3n + 2) = 3n^2 + 4n + 2$.
- Costo total: $3n^2 + 4n + 6$.

Tiempo de ejecución de un Algoritmo

¿Cuál es la complejidad del siguiente algoritmo?

```
1  int suma(int n) {
2          int sum = 0, i, j;
3          for (i=0; i < n; i++)
4                  for (j=0; j < n; j++)
5                          sum++;
6          return sum;
7  }
```

- Orden del número de iteraciones: n^2.
  - Costo L.2: 3.
  - Costo cada for (L.3, L.4): $1 + (n + 1) + n = 2n + 2$.
  - Costo L.5: 1.
  - Costo L.6: 1.
- Costo for interno (L.4 + L.5): $2n + 2 + n = 3n + 2$.
- Costo de doble for (L.3 + L.4 + L.5): $(2n + 2) + n \times (3n + 2) = 3n^2 + 4n + 2$.
- Costo total: $3n^2 + 4n + 6$.

Tiempo de ejecución de un Algoritmo

¿Cuál es la complejidad del siguiente algoritmo?

```
1  int suma(int n) {
2          int sum = 0, i, j;
3          for (i=0; i < n; i++)
4                  for (j=0; j < n; j++)
5                          sum++;
6          return sum;
7  }
```

- Orden del número de iteraciones: n^2.
  - Costo L.2: 3.
  - Costo cada for (L.3, L.4): $1 + (n + 1) + n = 2n + 2$.
  - Costo L.5: 1.
  - Costo L.6: 1.
- Costo for interno (L.4 + L.5): $2n + 2 + n = 3n + 2$.
- Costo de doble for (L.3 + L.4 + L.5): $(2n + 2) + n \times (3n + 2) = 3n^2 + 4n + 2$.
- Costo total: $3n^2 + 4n + 6$.

Tiempo de ejecución de un Algoritmo

¿Cuál es la complejidad del siguiente algoritmo?

```
1  int suma(int n) {
2          int sum = 0, i, j;
3          for (i=0; i < n; i++)
4                  for (j=0; j < n; j++)
5                          sum++;
6          return sum;
7  }
```

- Orden del número de iteraciones: n^2.
  - Costo L.2: 3.
  - Costo cada for (L.3, L.4): $1 + (n + 1) + n = 2n + 2$.
  - Costo L.5: 1.
  - Costo L.6: 1.
- Costo for interno (L.4 + L.5): $2n + 2 + n = 3n + 2$.
- Costo de doble for (L.3 + L.4 + L.5): $(2n + 2) + n \times (3n + 2) = 3n^2 + 4n + 2$.
- Costo total: $3n^2 + 4n + 6$.

Tiempo de ejecución de un Algoritmo

¿Cuál es la complejidad del siguiente algoritmo?

```
1  int suma(int n) {
2          int sum = 0, i, j;
3          for (i=0; i < n; i++)
4                  for (j=0; j < n; j++)
5                          sum++;
6          return sum;
7  }
```

- Orden del número de iteraciones: n^2.
  - Costo L.2: 3.
  - Costo cada for (L.3, L.4): $1 + (n + 1) + n = 2n + 2$.
  - Costo L.5: 1.
  - Costo L.6: 1.
- Costo for interno (L.4 + L.5): $2n + 2 + n = 3n + 2$.
- Costo de doble for (L.3 + L.4 + L.5): $(2n + 2) + n \times (3n + 2) = 3n^2 + 4n + 2$.
- Costo total: $3n^2 + 4n + 6$.

Tiempo de ejecución de un Algoritmo

¿Cuál es la complejidad del siguiente algoritmo?

```
1  int suma(int n) {
2          int sum = 0, i, j;
3          for (i=0; i < n; i++)
4                  for (j=0; j < n; j++)
5                          sum++;
6          return sum;
7  }
```

- Orden del número de iteraciones: n^2.
  - Costo L.2: 3.
  - Costo cada for (L.3, L.4): $1 + (n + 1) + n = 2n + 2$.
  - Costo L.5: 1.
  - Costo L.6: 1.
- Costo for interno (L.4 + L.5): $2n + 2 + n = 3n + 2$.
- Costo de doble for (L.3 + L.4 + L.5): $(2n + 2) + n \times (3n + 2) = 3n^2 + 4n + 2$.
- Costo total: $3n^2 + 4n + 6$.

Tiempo de ejecución de un Algoritmo

¿Cuál es la complejidad del siguiente algoritmo?

```
1  int suma(int n) {
2          int sum = 0, i, j;
3          for (i=0; i < n; i++)
4                  for (j=0; j < n; j++)
5                          sum++;
6          return sum;
7  }
```

- Orden del número de iteraciones: n^2.
  - Costo L.2: 3.
  - Costo cada for (L.3, L.4): $1 + (n + 1) + n = 2n + 2$.
  - Costo L.5: 1.
  - Costo L.6: 1.
- Costo for interno (L.4 + L.5): $2n + 2 + n = 3n + 2$.
- Costo de doble for (L.3 + L.4 + L.5): $(2n + 2) + n \times (3n + 2) = 3n^2 + 4n + 2$.
- Costo total: $3n^2 + 4n + 6$.

Tiempo de ejecución de un Algoritmo

¿Cuál es la complejidad del siguiente algoritmo?

```
1  int suma(int n) {
2          int sum = 0, i, j;
3          for (i=0; i < n; i++)
4                  for (j=0; j < n; j++)
5                          sum++;
6          return sum;
7  }
```

- Orden del número de iteraciones: n^2.
  - Costo L.2: 3.
  - Costo cada for (L.3, L.4): $1 + (n + 1) + n = 2n + 2$.
  - Costo L.5: 1.
  - Costo L.6: 1.
- Costo for interno (L.4 + L.5): $2n + 2 + n = 3n + 2$.
- Costo de doble for (L.3 + L.4 + L.5): $(2n + 2) + n \times (3n + 2) = 3n^2 + 4n + 2$.
- Costo total: $3n^2 + 4n + 6$.

Tiempo de ejecución de un Algoritmo

¿Cuál es la complejidad del siguiente algoritmo?

```
1  int suma(int n) {
2          int sum = 0, i, j;
3          for (i=0; i < n; i++)
4                  for (j=0; j < n; j++)
5                          sum++;
6          return sum;
7  }
```

- Orden del número de iteraciones: n^2.
  - Costo L.2: 3.
  - Costo cada for (L.3, L.4): $1 + (n + 1) + n = 2n + 2$.
  - Costo L.5: 1.
  - Costo L.6: 1.
- Costo for interno (L.4 + L.5): $2n + 2 + n = 3n + 2$.
- Costo de doble for (L.3 + L.4 + L.5): $(2n + 2) + n \times (3n + 2) = 3n^2 + 4n + 2$.
- Costo total: $3n^2 + 4n + 6$.

Tiempo de ejecución de un Algoritmo

¿Cuál es la complejidad del siguiente algoritmo?

```
1  int suma(int n) {
2          int sum = 0, i, j;
3          for (i=0; i < n; i++)
4                  for (j=0; j < n; j++)
5                          sum++;
6          return sum;
7  }
```

- Orden del número de iteraciones: n^2.
  - Costo L.2: 3.
  - Costo cada for (L.3, L.4): $1 + (n + 1) + n = 2n + 2$.
  - Costo L.5: 1.
  - Costo L.6: 1.
- Costo for interno (L.4 + L.5): $2n + 2 + n = 3n + 2$.
- Costo de doble for (L.3 + L.4 + L.5): $(2n + 2) + n \times (3n + 2) = 3n^2 + 4n + 2$.
- Costo total: $3n^2 + 4n + 6$.

Tiempo de ejecución de un Algoritmo

¿Cuál es la complejidad del siguiente algoritmo?

```
1  int suma(int n) {
2          int sum = 0, i, j;
3          for (i=0; i < n; i++)
4                  for (j=0; j < n; j++)
5                          sum++;
6          return sum;
7  }
```

- Orden del número de iteraciones: n^2.
  - Costo L.2: 3.
  - Costo cada for (L.3, L.4): $1 + (n + 1) + n = 2n + 2$.
  - Costo L.5: 1.
  - Costo L.6: 1.
- Costo for interno (L.4 + L.5): $2n + 2 + n = 3n + 2$.
- Costo de doble for (L.3 + L.4 + L.5): $(2n + 2) + n \times (3n + 2) = 3n^2 + 4n + 2$.
- Costo total: $3n^2 + 4n + 6$.

Tiempo de ejecución de un Algoritmo

¿Cuál es la complejidad del siguiente algoritmo?

```
1  int suma(int n) {
2          int sum = 0, i, j;
3          for (i=0; i < n; i++)
4                  for (j=0; j < n; j++)
5                          sum++;
6          return sum;
7  }
```

- Orden del número de iteraciones: n^2.
  - Costo L.2: 3.
  - Costo cada for (L.3, L.4): $1 + (n + 1) + n = 2n + 2$.
  - Costo L.5: 1.
  - Costo L.6: 1.
- Costo for interno (L.4 + L.5): $2n + 2 + n = 3n + 2$.
- Costo de doble for (L.3 + L.4 + L.5): $(2n + 2) + n \times (3n + 2) = 3n^2 + 4n + 2$.
- Costo total: $3n^2 + 4n + 6$.

Complejidad Algorítmica vs Entorno de ejecución.

En esta unidad veremos algoritmos de ordenamiento, los primeros dos que veremos (más adelante) son:

- **Insert Sort**: $T_1(n) = c_1 \times n^2$
- **Merge Sort**: $T_2(n) = c_2 \times n \times \log_2 n$

Supongamos que el PC-A está ejecutando **Insert Sort**, que el PC-B está ejecutando **Merge Sort** y que ambos deben ordenar un arreglo de 10 millones de enteros. Además, el PC-A es ultima generación y ejecuta 10 mil millones de instrucciones por segundo y el PC-B tiene 10 años de antigüedad, ergo solo procesa 10 millones de instrucciones por segundo. Incluyamos lo siguiente:

- Paco Web programó **Insert Sort** en el PC-A usando lenguaje de máquina (assembly) resultando en un costo de $2 \times n^2$ instrucciones para ordenar n números.
- El Corxea implementa **Merge Sort**, en el PC-B usando C++, con un compilador antiguo e ineficiente, y el código resultante toma $50 \times n \times \log_2 n$ instrucciones para ordenar n números.

Complejidad Algorítmica vs Entorno de ejecución.

- PC-A:

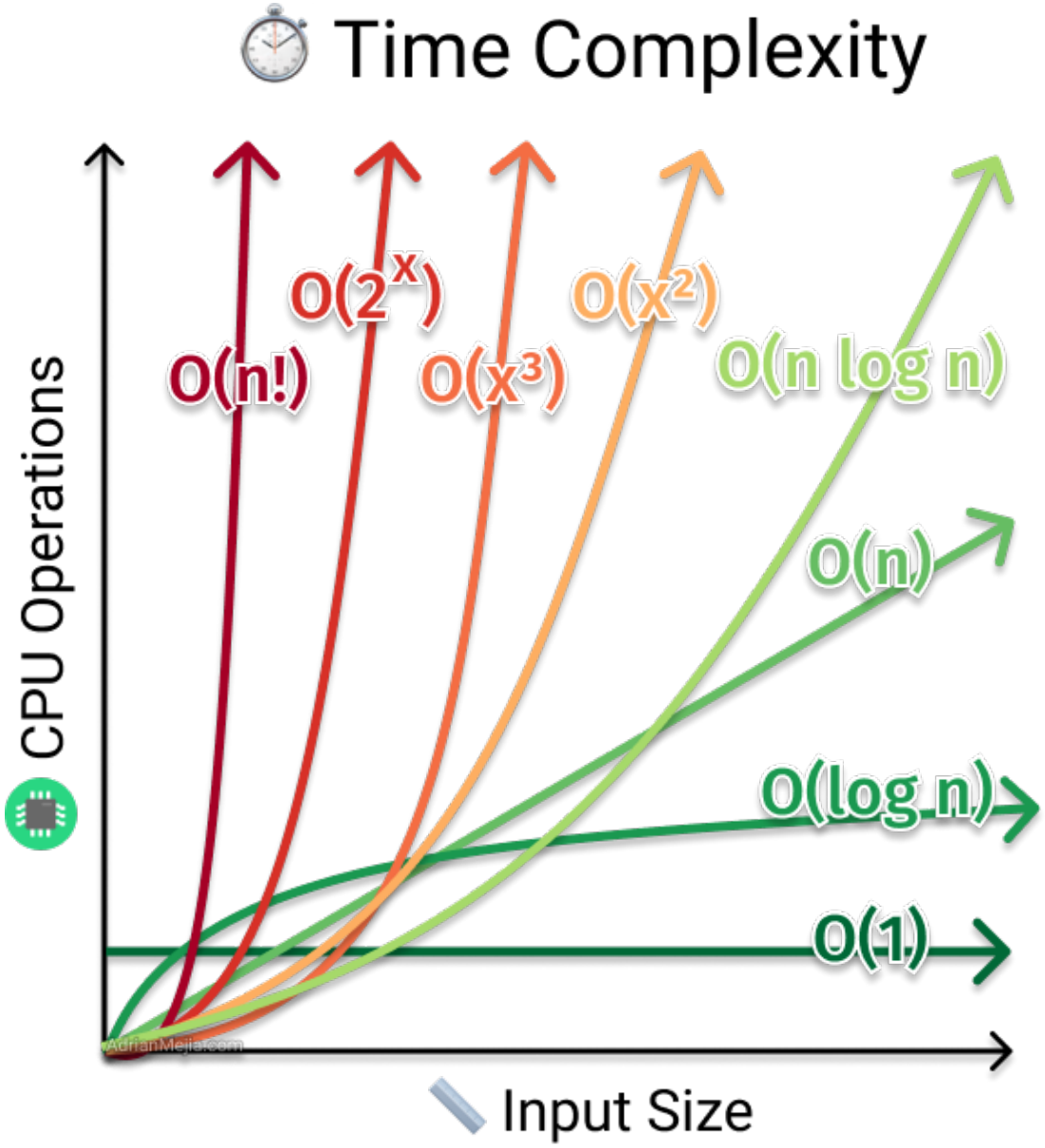
$$\frac{2 \times (10^7)^2}{10^{10}} = 20000\ [s] \quad (> 5{,}5[h])$$

- PC-B:

$$\frac{50 \times 10^7 \times \log_2 10^7}{10^7} \approx 1163\ [s] \quad (< 20[min])$$

PC-B completa su algoritmo 17 veces más rápido que el algoritmo usado en PC-A, siendo el primer entorno de ejecución 1000 veces más lento que el segundo.

Complejidad Algorítmica vs Entorno de ejecución.



Análisis Asintótico

Una vez entendido el análisis de tiempo buscaremos análisis asintótico de los algoritmos. En la práctica significa concentrarse en el término predominante en $T(n)$, y luego encontrar cotas superiores y/o inferiores de rendimiento.
Existen tres notaciones asintóticas generalmente usadas:

- Big O ($\mathcal{O}$): Cota superior asintótica, vale decir, cota al peor desempeño.
- Big Omega (Ω): Cota inferior asintótica, vale decir, cota al mejor desempeño.
- Big Theta (Θ): Cota compuesta, vale decir, cota al rendimiento promedio.

Comúnmente, se utiliza la notación en peor escenario, vale decir, Big O ($\mathcal{O}$).

Merge Sort

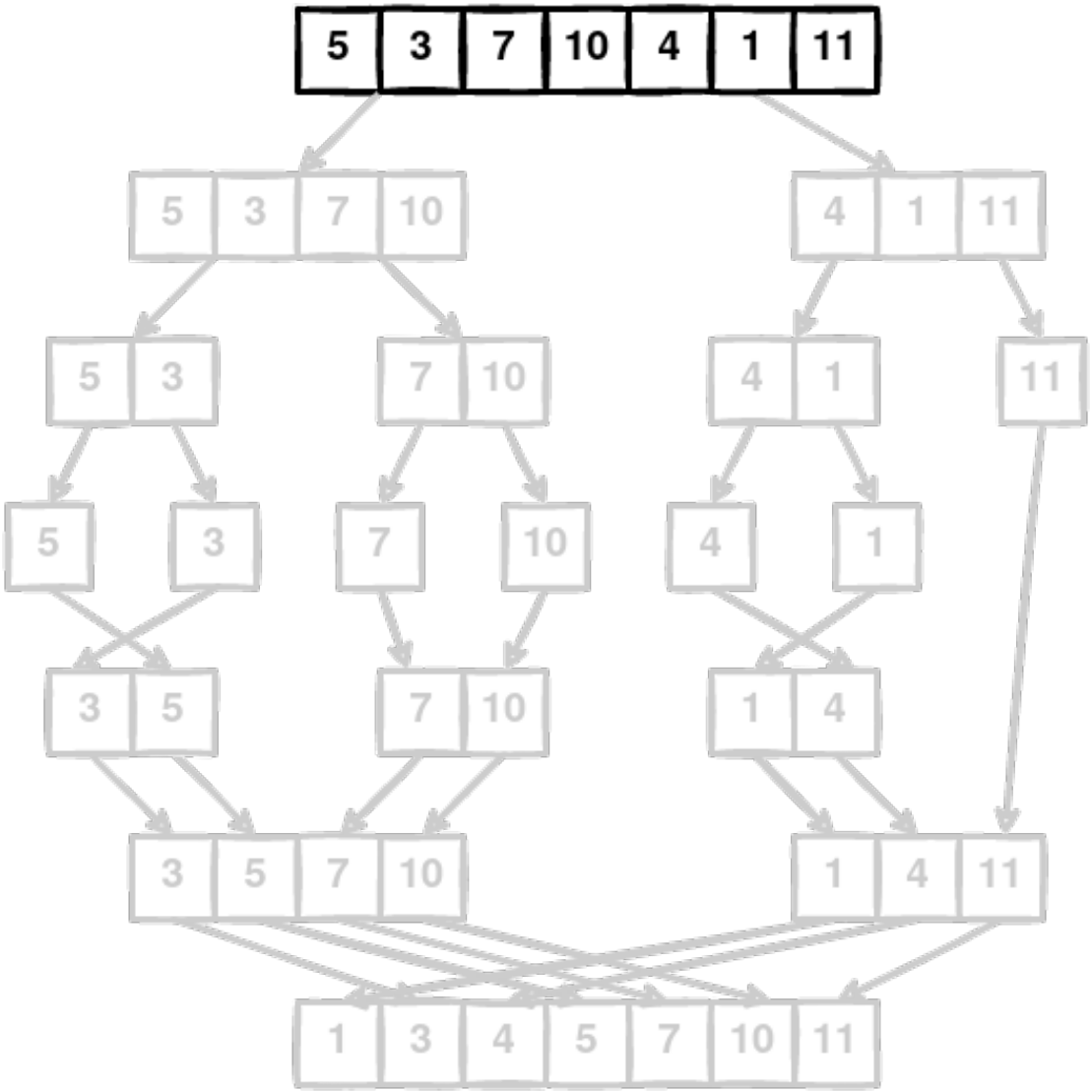
Algoritmo de ordenamiento **recursivo** que aplica la técnica **Divide & Conquer**.

- Peor escenario: $\mathcal{O}(n \log n)$
- Mejor escenario: $\Omega(n \log n)$
- Escenario Promedio: $\Theta(n \log n)$

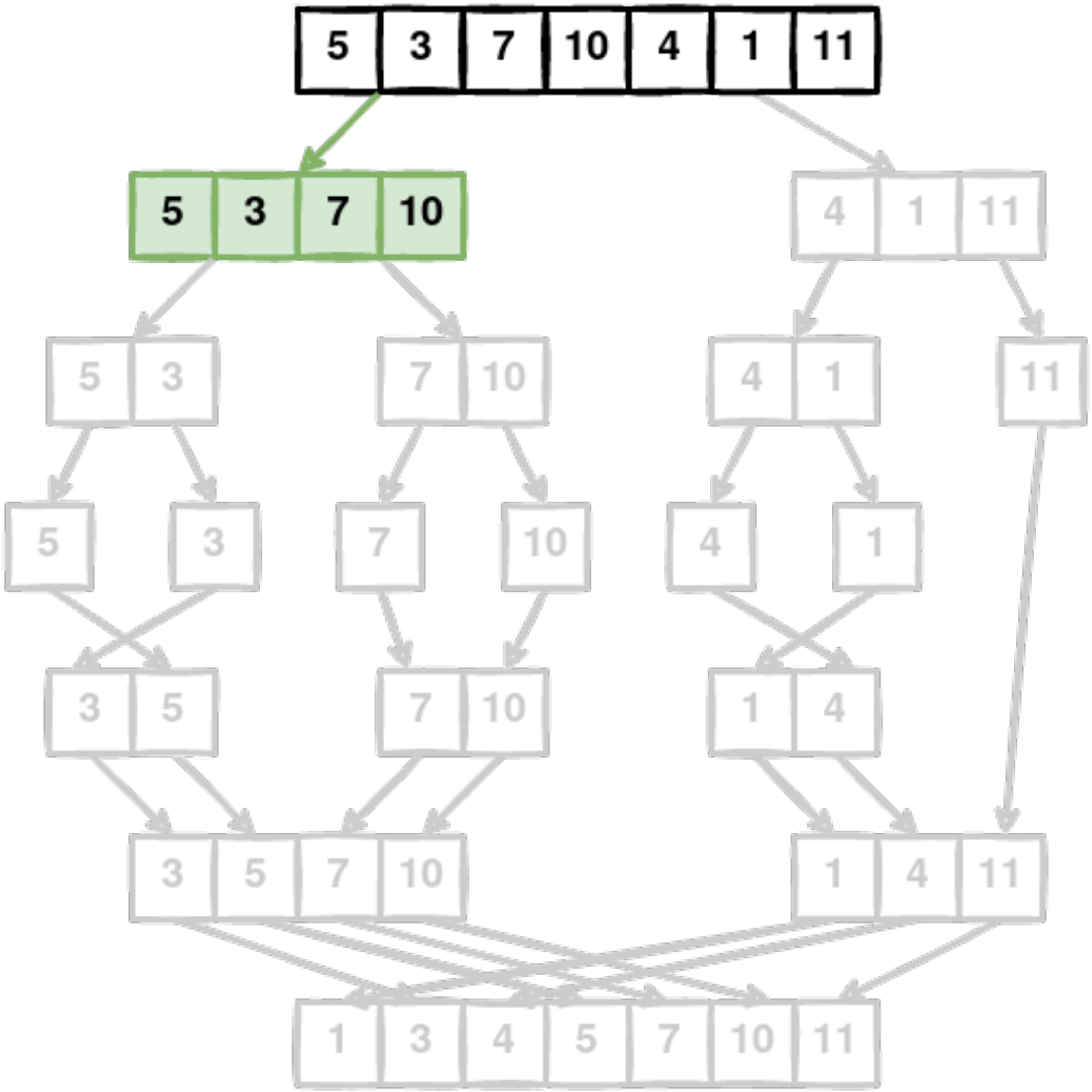
Características:

- Estable y rápido, sobre todo para tamaños grandes de datos.
- Es superado en tamaños pequeños de datos.
- Necesita un espacio de memoria extra para el ordenamiento ($\Theta(n)$).

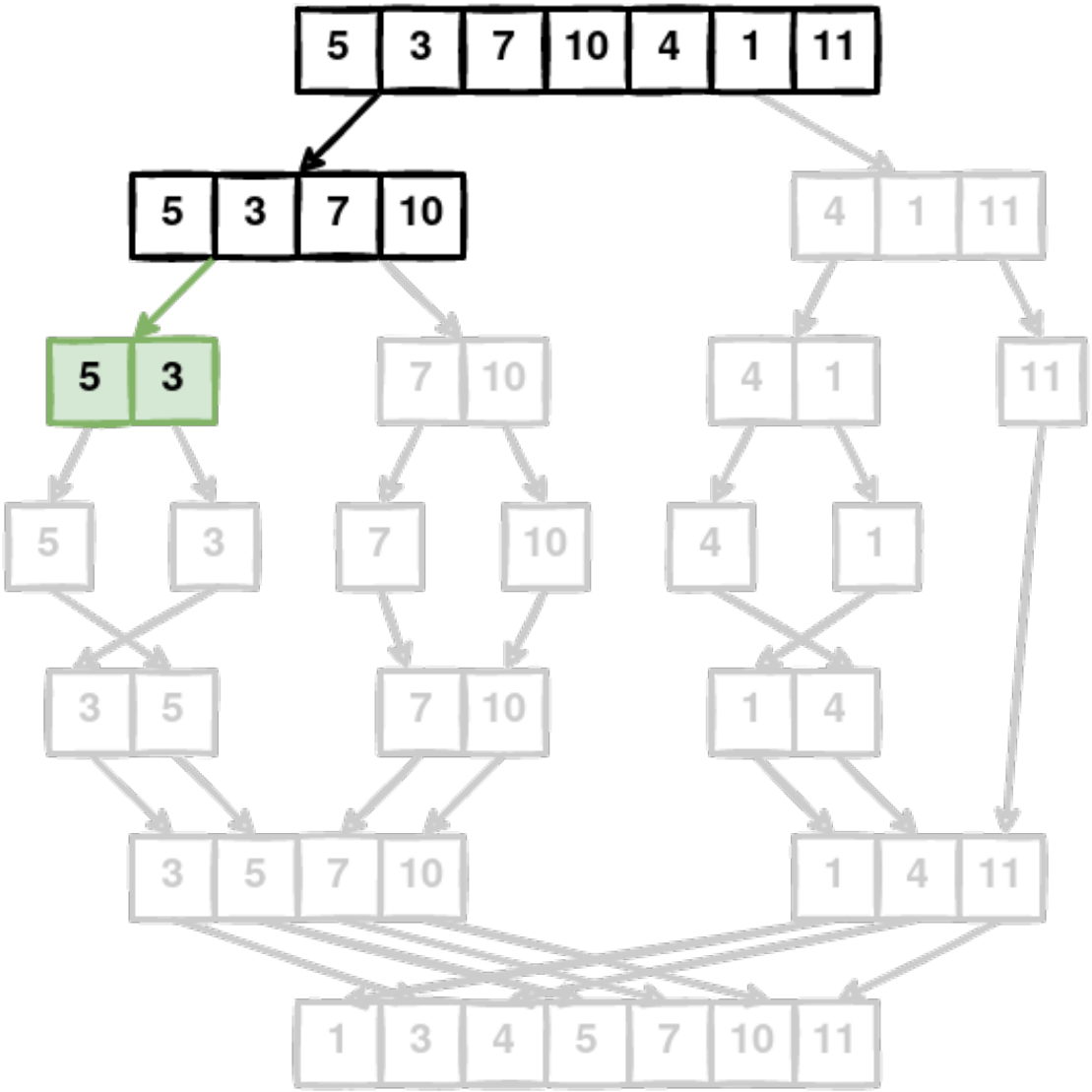
Merge Sort


5 3 7 10 4 1 11
5 3 7 10
4 1 11
5 3
7 10
4 1
11
5
3
7
10
4
1
3 5
7 10
1 4
3 5 7 10
1 4 11
1 3 4 5 7 10 11

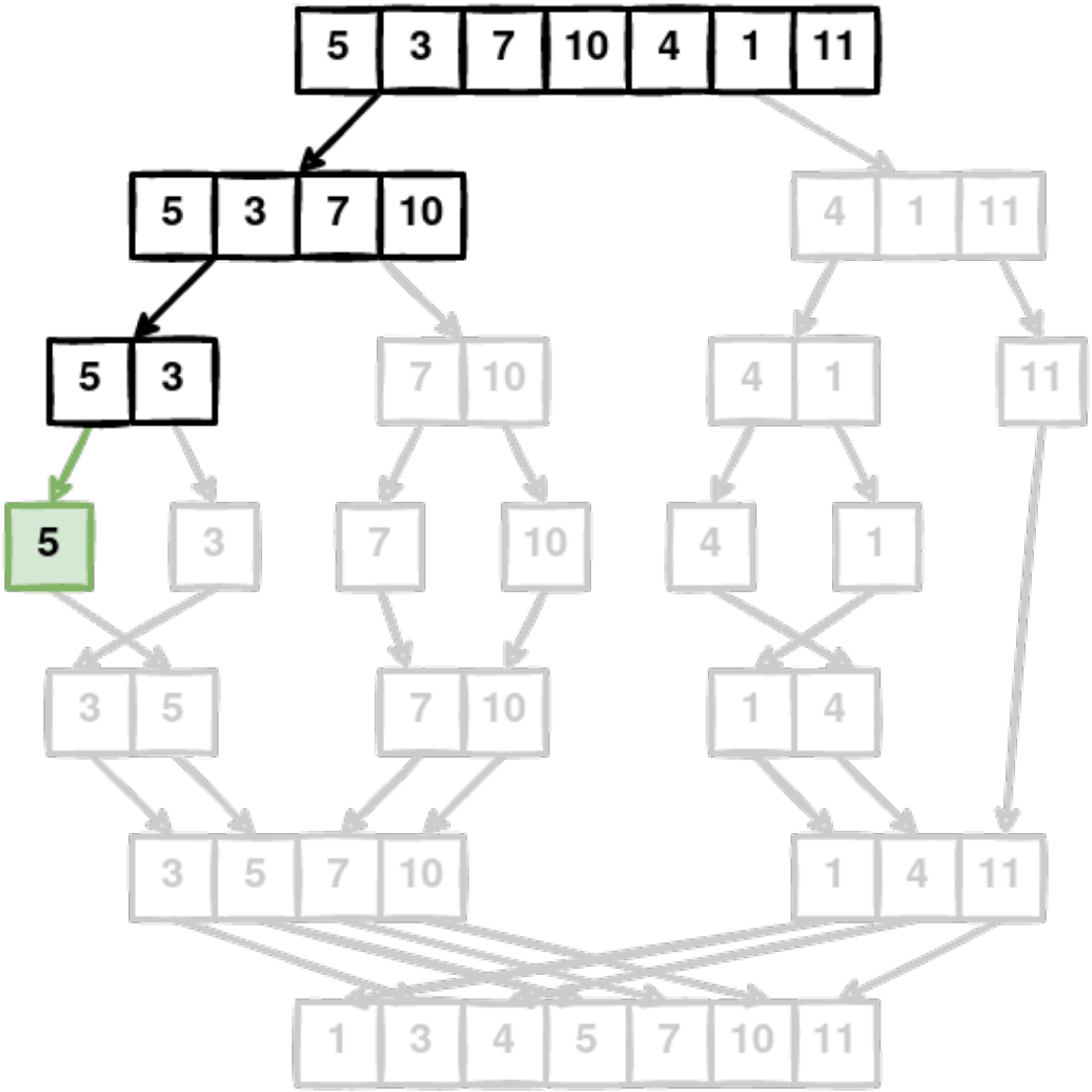
Merge Sort


5 3 7 10 4 1 11
5 3 7 10
4 1 11
5 3
7 10
4 1
11
5
3
7
10
4
1
3 5
7 10
1 4
3 5 7 10
1 4 11
1 3 4 5 7 10 11

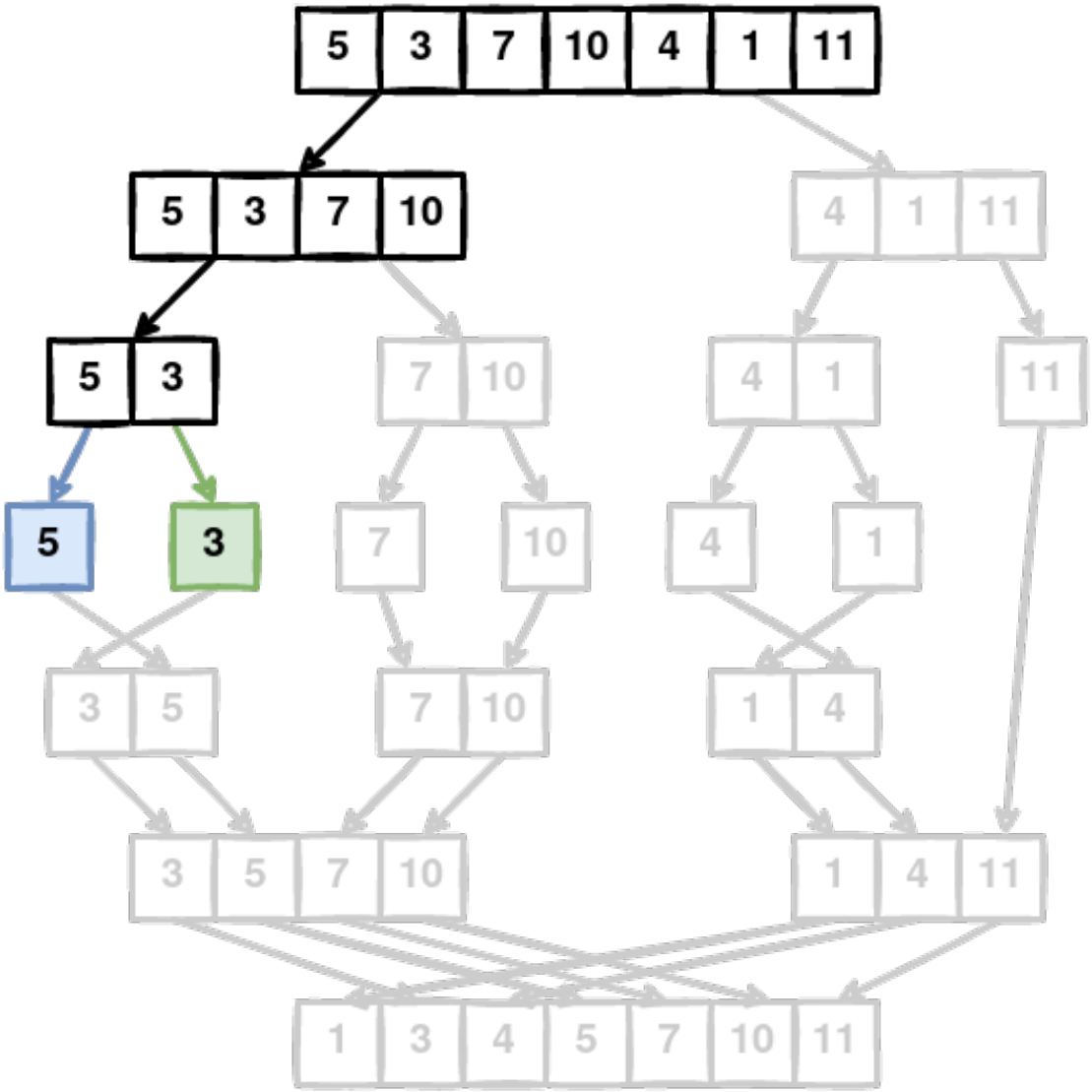
Merge Sort


5 3 7 10 4 1 11
5 3 7 10
4 1 11
5 3
7 10
4 1
11
5
3
7
10
4
1
3 5
7 10
1 4
3 5 7 10
1 4 11
1 3 4 5 7 10 11

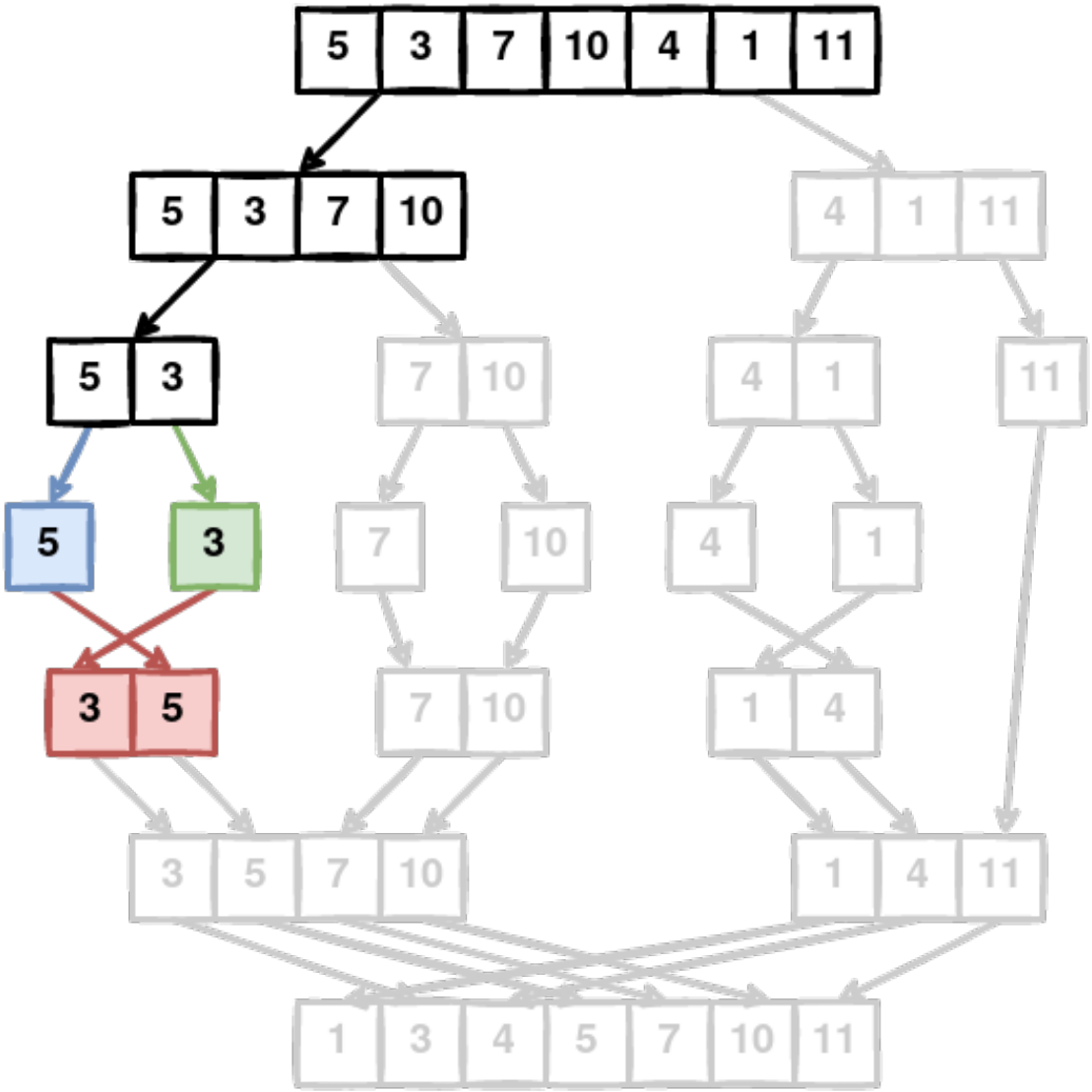
Merge Sort


5 3 7 10 4 1 11
5 3 7 10
4 1 11
5 3
7 10
4 1
11
5
3
7
10
4
1
3 5
7 10
1 4
3 5 7 10
1 4 11
1 3 4 5 7 10 11

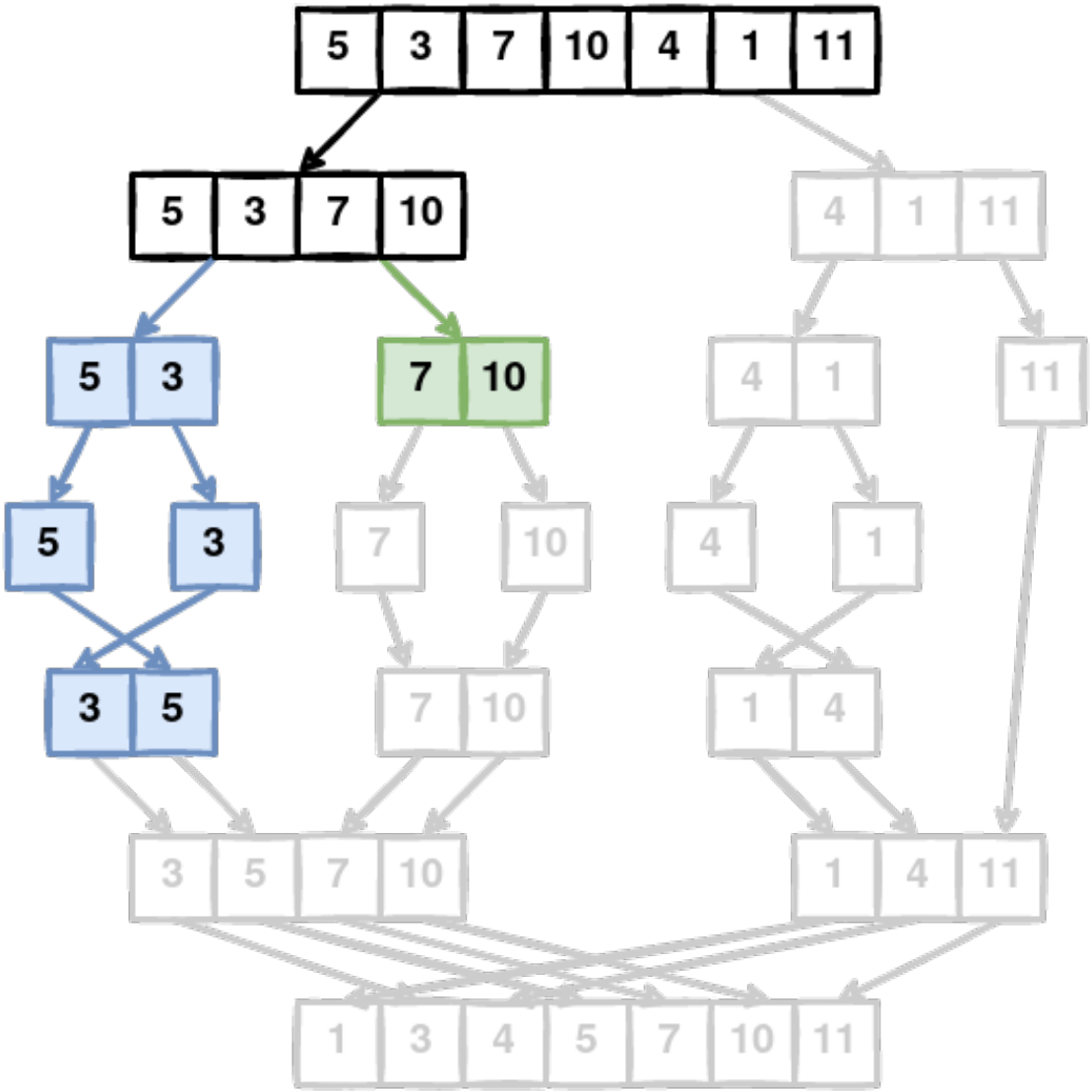
Merge Sort


5 3 7 10 4 1 11
5 3 7 10
4 1 11
5 3
7 10
4 1
11
5
3
7
10
4
1
3 5
7 10
1 4
3 5 7 10
1 4 11
1 3 4 5 7 10 11

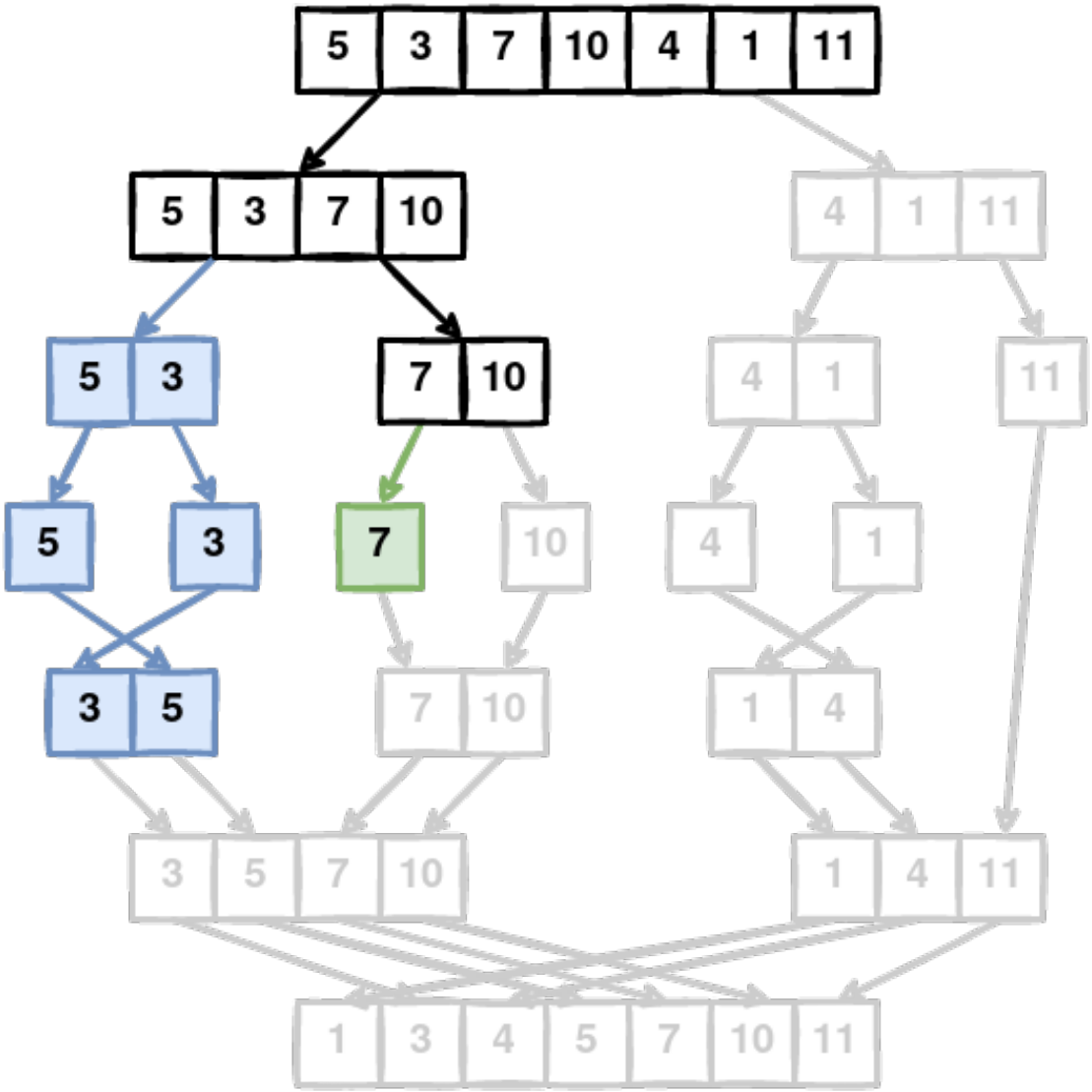
Merge Sort


5 3 7 10 4 1 11
5 3 7 10
4 1 11
5 3
7 10
4 1
11
5
3
7
10
4
1
3 5
7 10
1 4
3 5 7 10
1 4 11
1 3 4 5 7 10 11

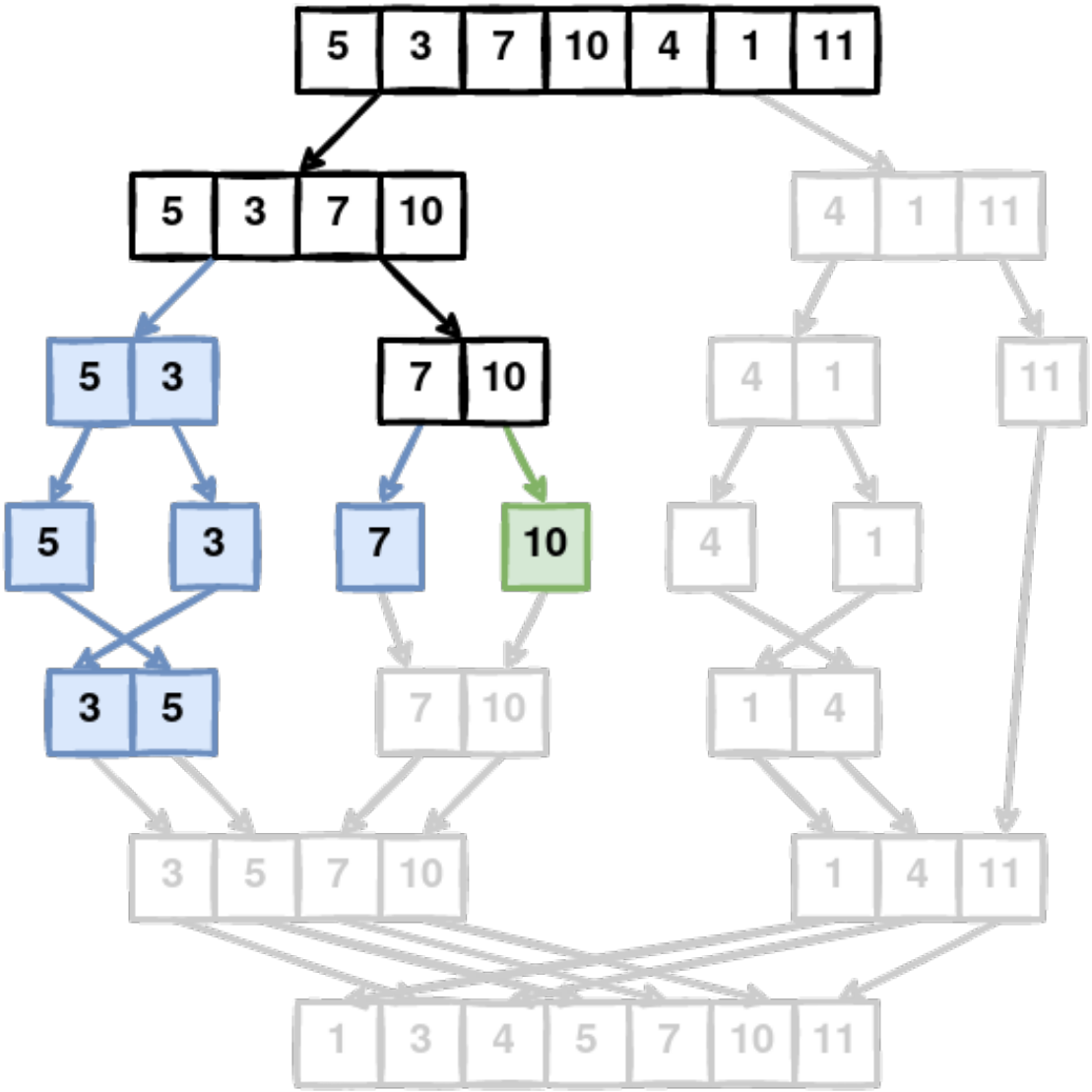
Merge Sort


5 3 7 10 4 1 11
5 3 7 10
4 1 11
5 3
7 10
4 1
11
5
3
7
10
4
1
3 5
7 10
1 4
3 5 7 10
1 4 11
1 3 4 5 7 10 11

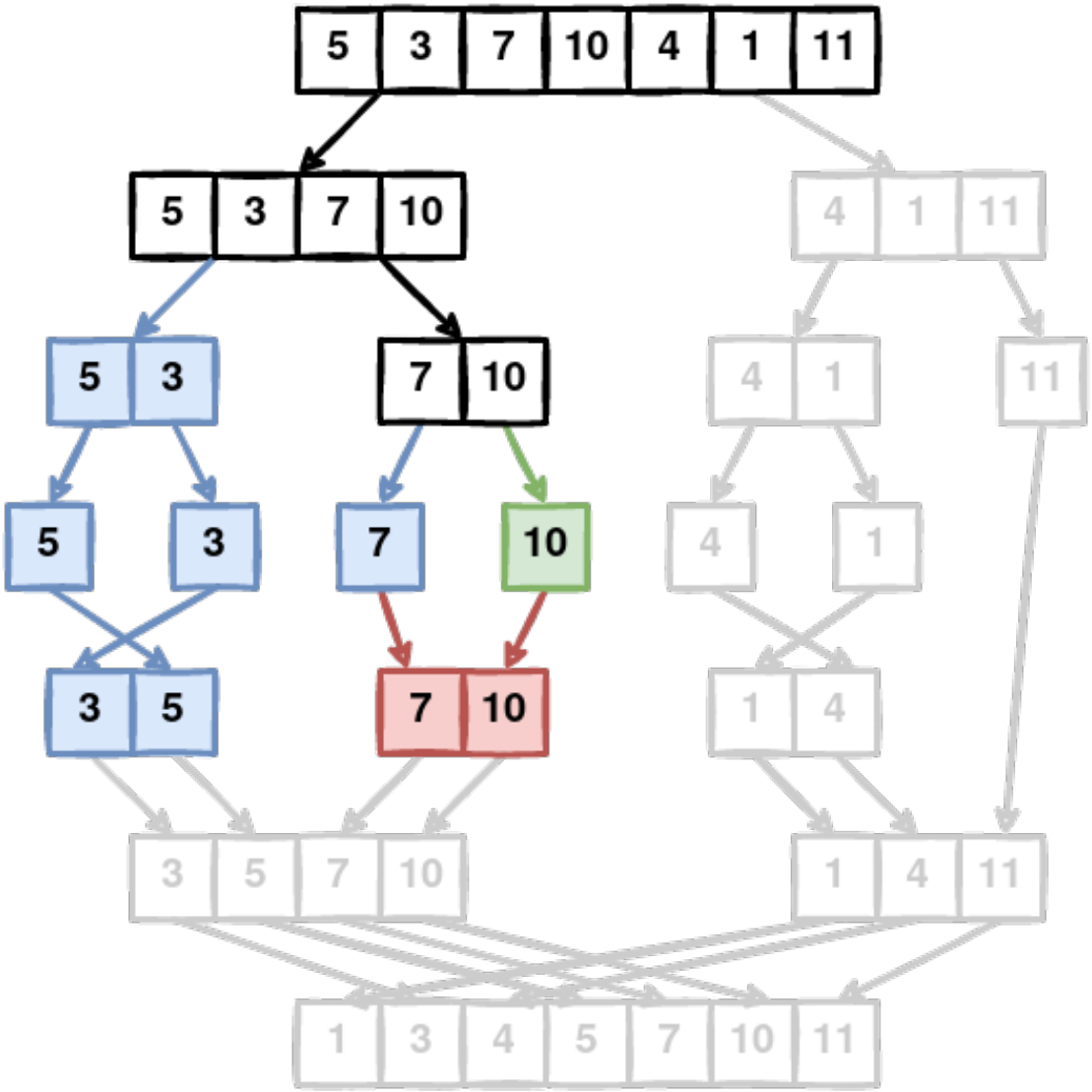
Merge Sort


5 3 7 10 4 1 11
5 3 7 10
4 1 11
5 3
7 10
4 1
11
5
3
7
10
4
1
3 5
7 10
1 4
3 5 7 10
1 4 11
1 3 4 5 7 10 11

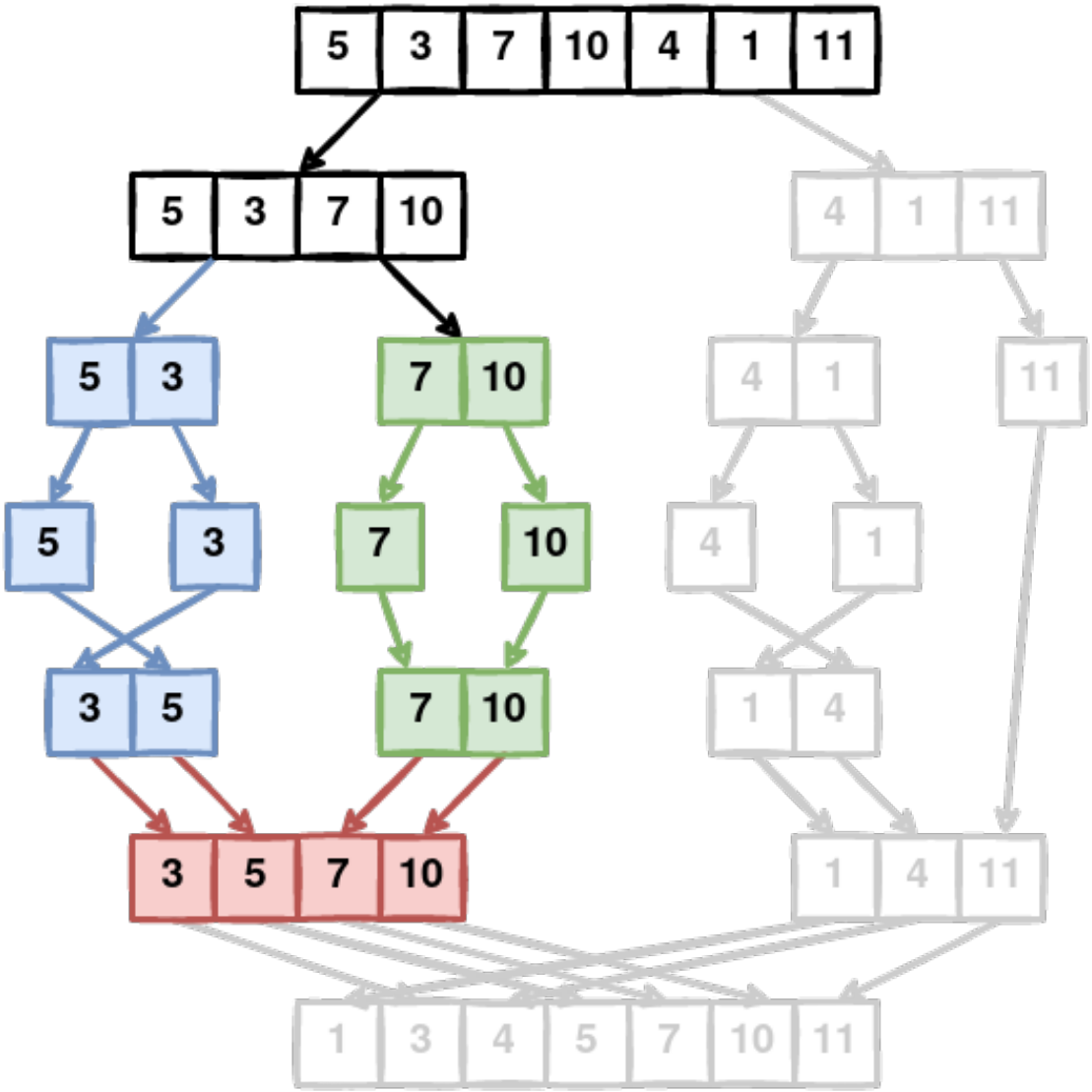
Merge Sort


5 3 7 10 4 1 11
5 3 7 10
4 1 11
5 3
7 10
4 1
11
5
3
7
10
4
1
3 5
7 10
1 4
3 5 7 10
1 4 11
1 3 4 5 7 10 11

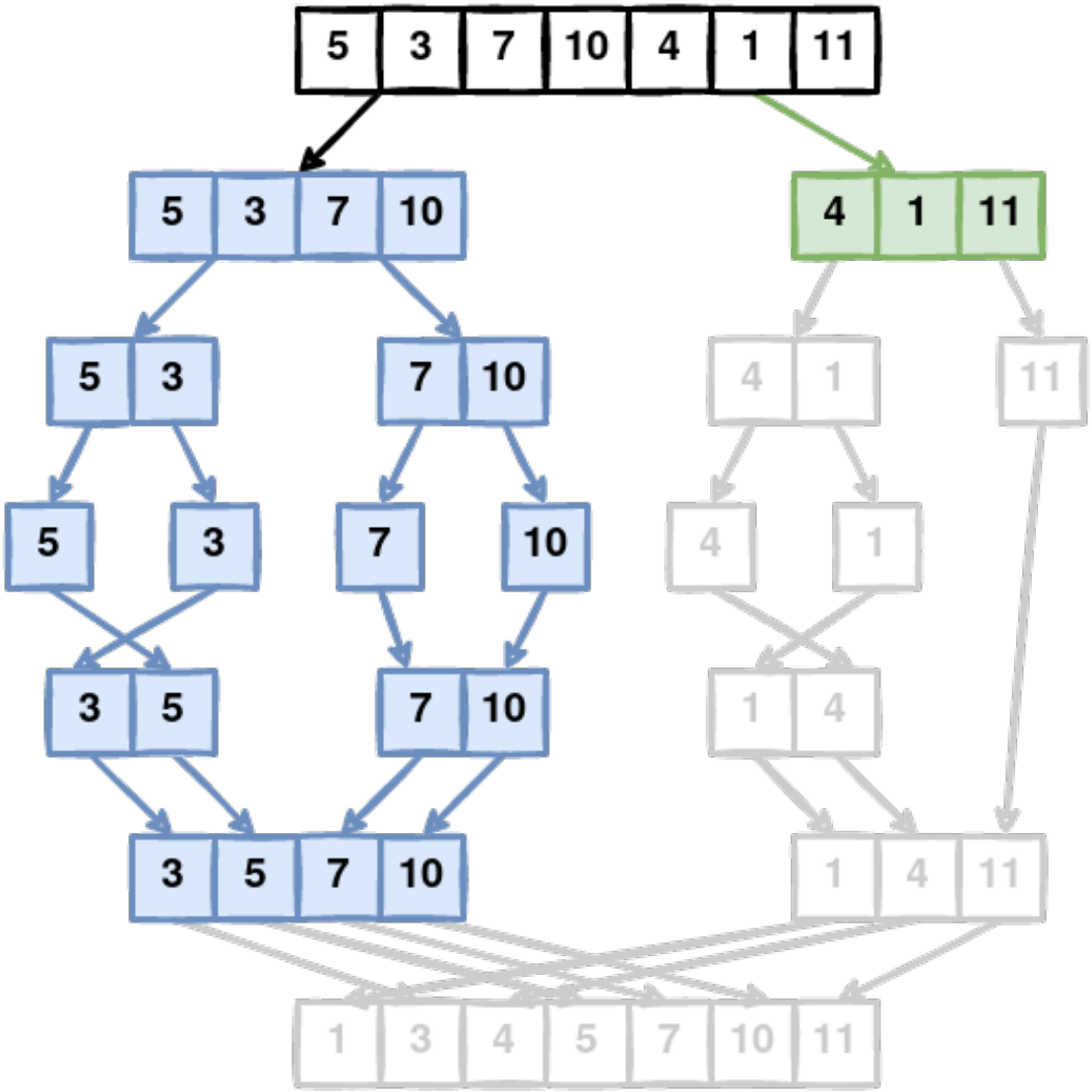
Merge Sort


5 3 7 10 4 1 11
5 3 7 10
4 1 11
5 3
7 10
4 1
11
5
3
7
10
4
1
3 5
7 10
1 4
3 5 7 10
1 4 11
1 3 4 5 7 10 11

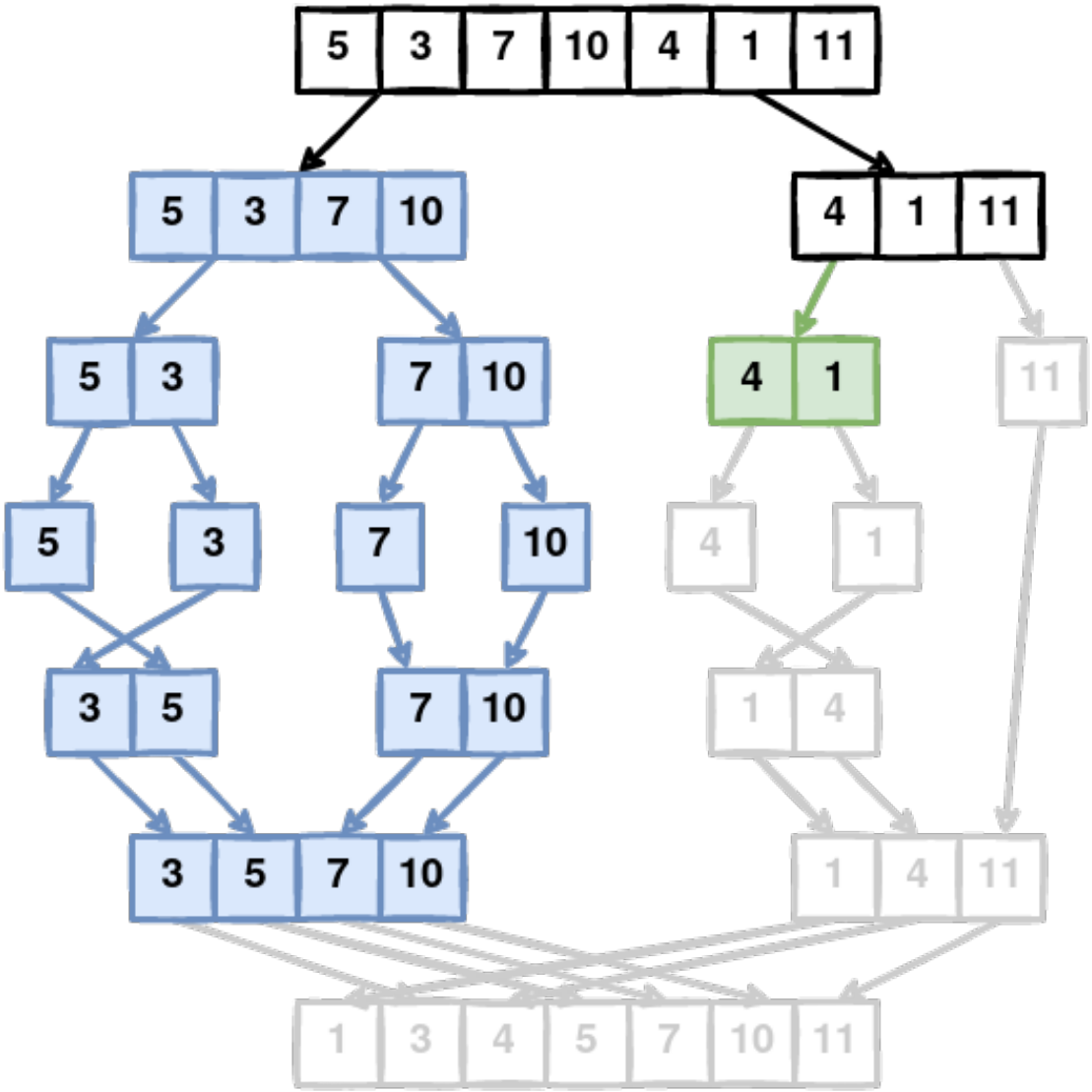
Merge Sort


5 3 7 10 4 1 11
5 3 7 10
4 1 11
5 3
7 10
4 1
11
5
3
7
10
4
1
3 5
7 10
1 4
3 5 7 10
1 4 11
1 3 4 5 7 10 11

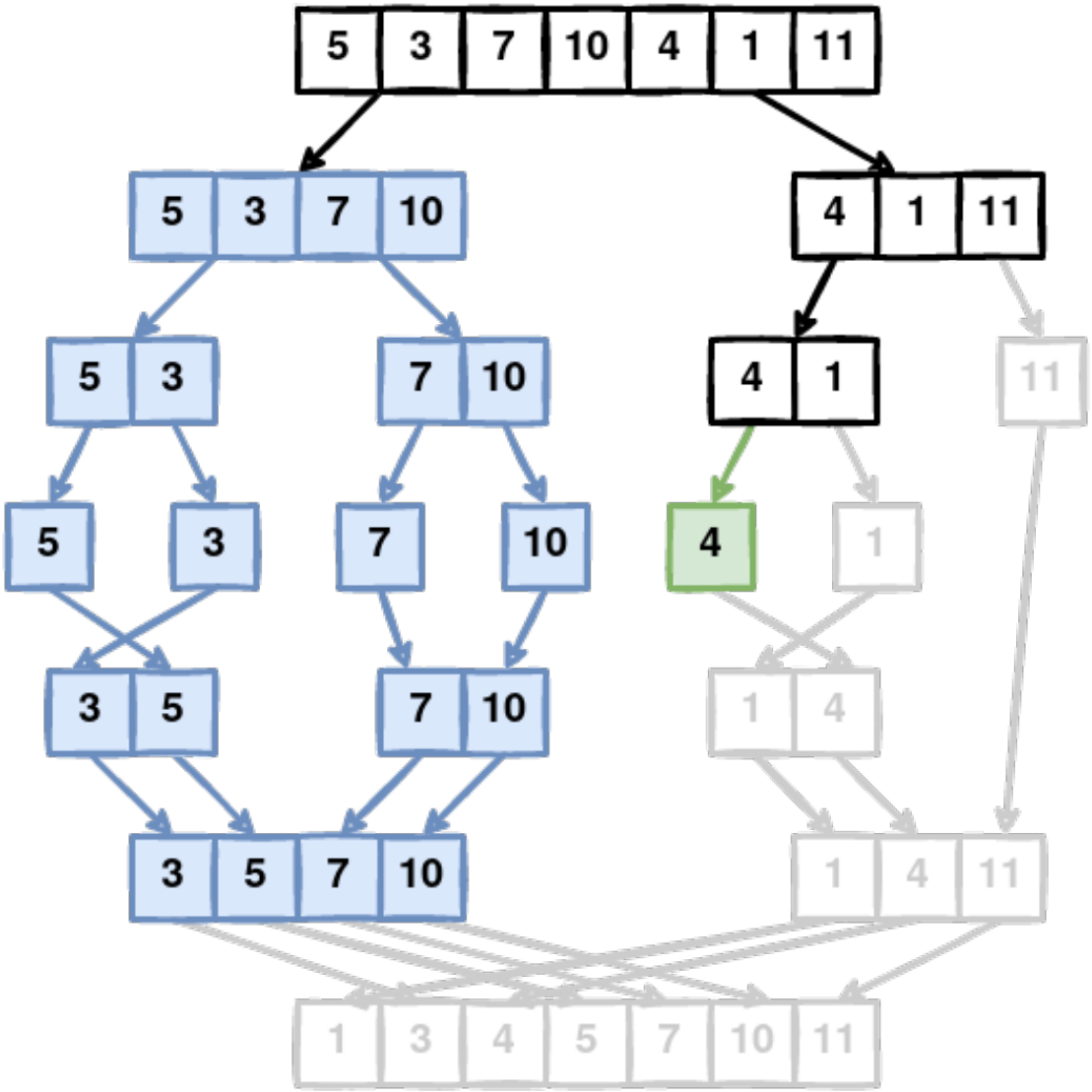
Merge Sort


5 3 7 10 4 1 11
5 3 7 10
4 1 11
5 3
7 10
4 1
11
5
3
7
10
4
1
3 5
7 10
1 4
3 5 7 10
1 4 11
1 3 4 5 7 10 11

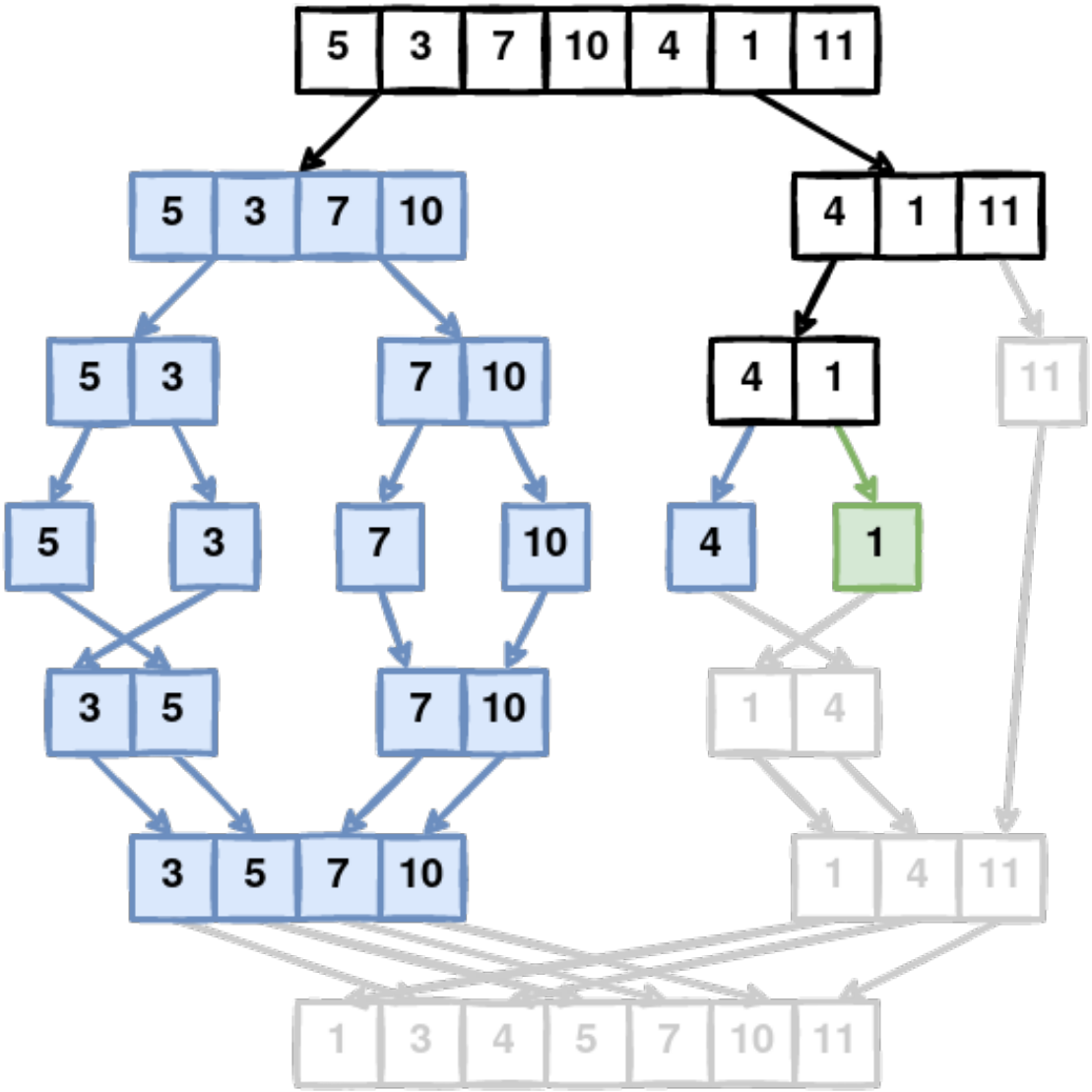
Merge Sort


5 3 7 10 4 1 11
5 3 7 10
4 1 11
5 3
7 10
4 1
11
5
3
7
10
4
1
3 5
7 10
1 4
3 5 7 10
1 4 11
1 3 4 5 7 10 11

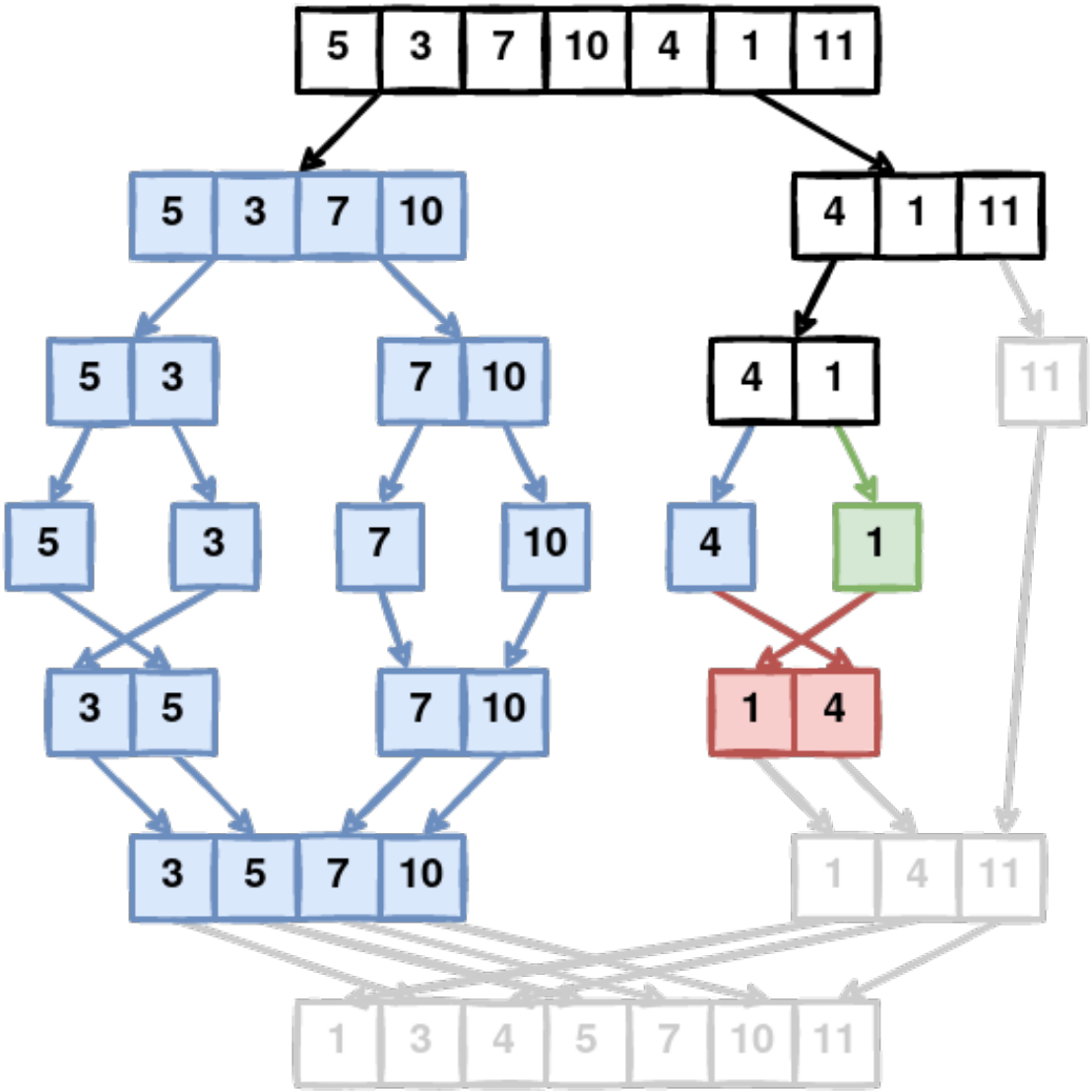
Merge Sort


5 3 7 10 4 1 11
5 3 7 10
4 1 11
5 3
7 10
4 1
11
5
3
7
10
4
1
3 5
7 10
1 4
3 5 7 10
1 4 11
1 3 4 5 7 10 11

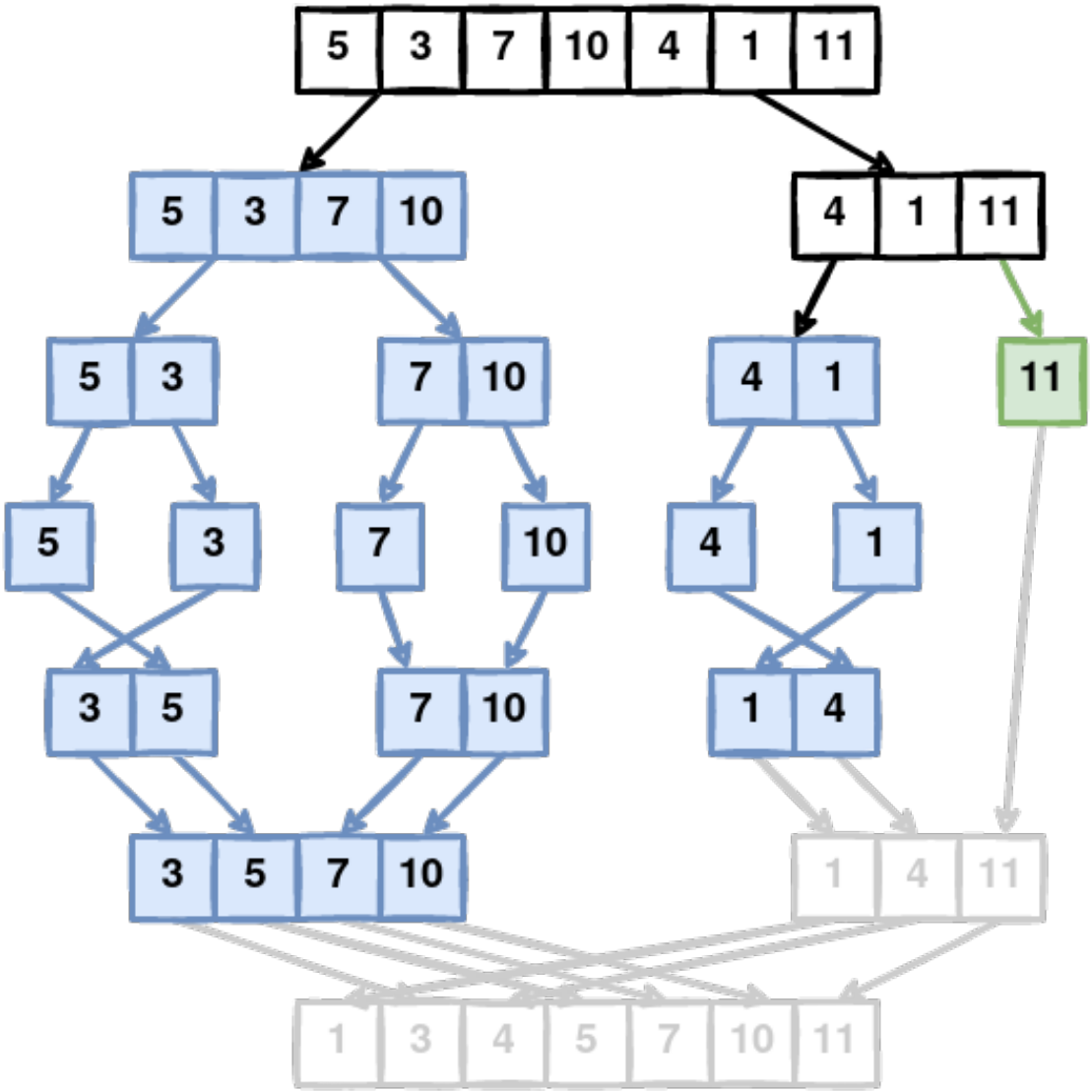
Merge Sort


5 3 7 10 4 1 11
5 3 7 10
4 1 11
5 3
7 10
4 1
11
5
3
7
10
4
1
3 5
7 10
1 4
3 5 7 10
1 4 11
1 3 4 5 7 10 11

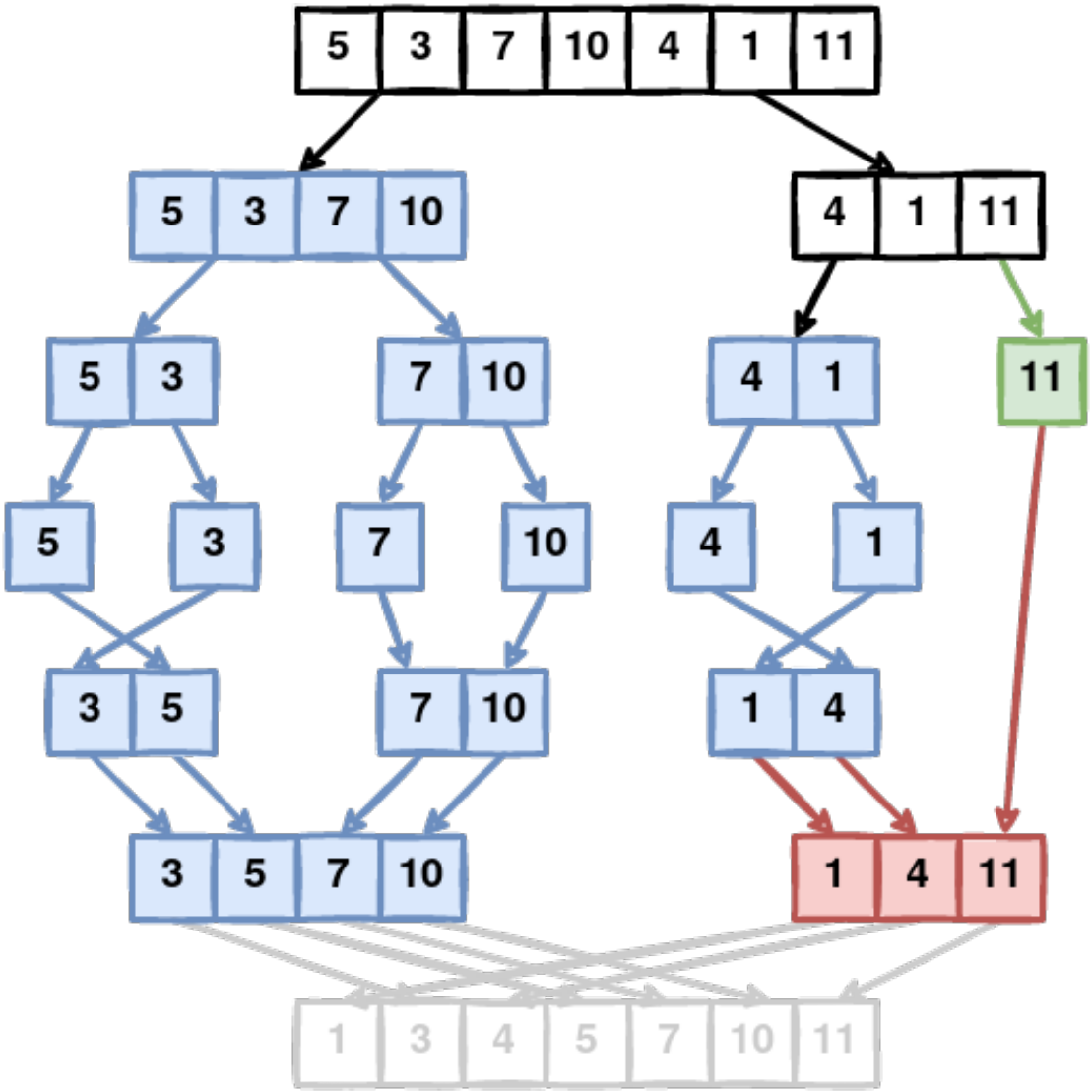
Merge Sort


5 3 7 10 4 1 11
5 3 7 10
4 1 11
5 3
7 10
4 1
11
5
3
7
10
4
1
3 5
7 10
1 4
3 5 7 10
1 4 11
1 3 4 5 7 10 11

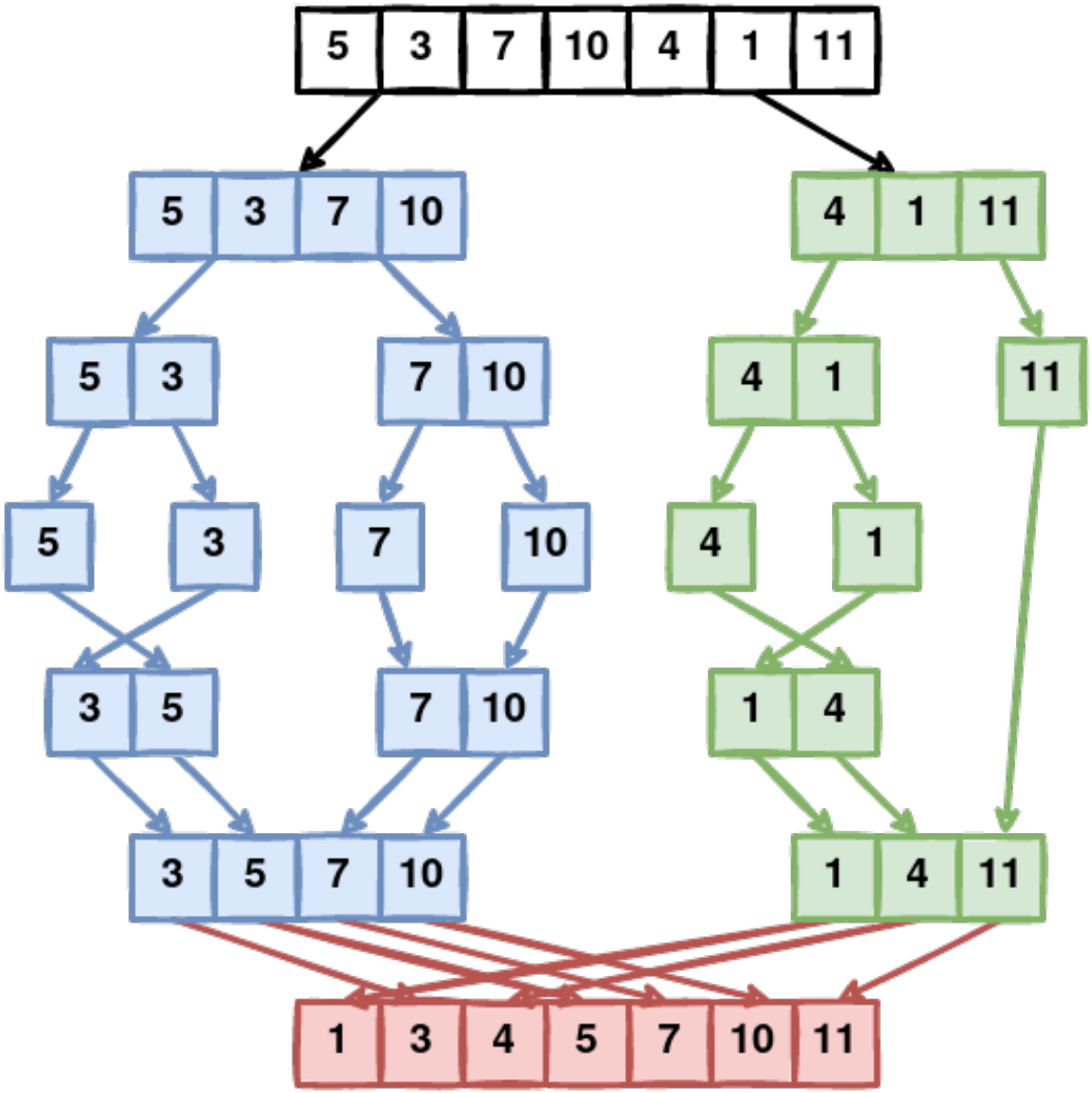
Merge Sort


5 3 7 10 4 1 11
5 3 7 10
4 1 11
5 3
7 10
4 1
11
5
3
7
10
4
1
3 5
7 10
1 4
3 5 7 10
1 4 11
1 3 4 5 7 10 11

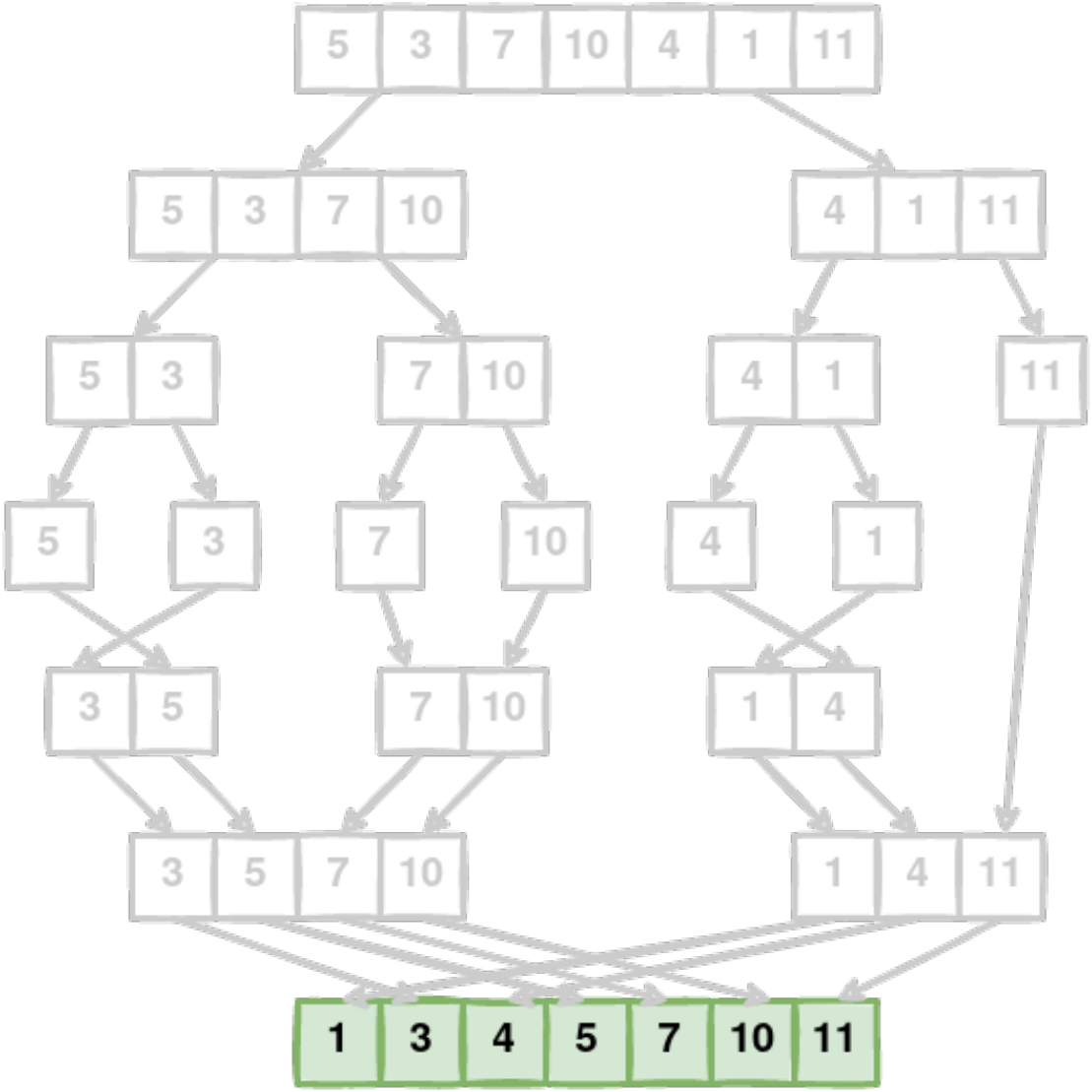
Merge Sort


5 3 7 10 4 1 11
5 3 7 10
4 1 11
5 3
7 10
4 1
11
5
3
7
10
4
1
3 5
7 10
1 4
3 5 7 10
1 4 11
1 3 4 5 7 10 11

Merge Sort


5 3 7 10 4 1 11
5 3 7 10
4 1 11
5 3
7 10
4 1
11
5
3
7
10
4
1
3 5
7 10
1 4
3 5 7 10
1 4 11
1 3 4 5 7 10 11

Merge Sort


5 3 7 10 4 1 11
5 3 7 10
4 1 11
5 3
7 10
4 1
11
5
3
7
10
4
1
3 5
7 10
1 4
3 5 7 10
1 4 11
1 3 4 5 7 10 11

MergeSort

MergeSort

```
void mergeSort(int arr[], int l, int r){ //l=izq, r=der (indices)
    if (l < r) {
        int m = l + (r - l) / 2; //div. entera

        mergeSort(arr, l, m); //dividir
        mergeSort(arr, m + 1, r); //post-orden

        merge(arr, l, m, r); //mezclar
    }
}
```

MergeSort

merge

```
void merge(int arr[], int l, int m, int r){
    int i, j, k;
    int n1 = m - l + 1;
    int n2 = r - m;

    int L[n1], R[n2]; //arreglos temporales

    for (i = 0; i < n1; i++) //copia a los arreglos
        L[i] = arr[l + i];
    for (j = 0; j < n2; j++)
        R[j] = arr[m + 1 + j];

    i = 0; j=0; k = l; //mezcla
    while (i<n1 && j<n2) {
        if (L[i] <= R[j])
            arr[k] = L[i]; i++;
        else
            arr[k] = R[j]; j++;
        k++;
    }

    while (i < n1) //si sobra de L
        arr[k] = L[i]; i++; k++;
    while (j < n2)  //si sobra de R
        arr[k] = R[j]; j++; k++;
}
```

Funciones de Recurrencia

¿Cómo podemos determinar la complejidad de MergeSort si éste trabaja recursivamente?

- Peor escenario: $\mathcal{O}(n \log n)$
- Mejor escenario: $\Omega(n \log n)$
- Escenario Promedio: $\Theta(n \log n)$

Funciones de recurrencia, relaciones matemáticas recursivas que son utilizadas en ciencias de la computación para explicar la complejidad de éste tipo de algoritmos.

Funciones de Recurrencia

Factorial

```
int factorial(int n){
        if(n>0)
                exit(-1);

        if(n==0)
                return 1;
        return n * factorial(n-1); // 1 + T(n-1)
}
```

Sabemos:

- $T(n) = 1 + T(n-1)$ (si $n > 0$); $T(n) = 1$ (si $n = 1$).
- Luego: $T(n) = 1 + 1 + 1 + \cdots + T(1) = n$.
- $T(n) = n \rightarrow \mathcal{O}(n)$.

Funciones de Recurrencia

Factorial

```
int factorial(int n){
        if(n>0)
                exit(-1);

        if(n==0)
                return 1;
        return n * factorial(n-1); // 1 + T(n-1)
}
```

Sabemos:

- $T(n) = 1 + T(n-1)$ (si $n > 0$); $T(n) = 1$ (si $n = 1$).
- Luego: $T(n) = 1 + 1 + 1 + \cdots + T(1) = n$.
- $T(n) = n \rightarrow \mathcal{O}(n)$.

Funciones de Recurrencia

Búsqueda Binaria

```
int binarySearch(int arr[], int l, int r, int x){
    if (r >= l) {
        int mid = l + (r - l) / 2;

        if (arr[mid] == x)
            return mid;

        if (arr[mid] > x) //if-else 1 + T(n/2)
            return binarySearch(arr, l, mid - 1, x);
        return binarySearch(arr, mid + 1, r, x); //else
    }
    return -1;
}
```

- $T(n) = 1 + T(n/2)$ (si $n > 1$); $T(n) = 1$ (si $n = 1$).
- Luego: $T(n) = 1 + 1 + 1 + \cdots + T(n/2^k)$
- Al finalizar: $T(n/2^k) = T(1) \rightarrow n = 2^k$
- $\log_2 n = log_2(2^k) \rightarrow k = \log_2 n$
- $T(n) = 1 + T(n/2) = 1 + \mathcal{O}(\log n) = \mathcal{O}(\log n)$

Funciones de Recurrencia

Búsqueda Binaria

```
int binarySearch(int arr[], int l, int r, int x){
    if (r >= l) {
        int mid = l + (r - l) / 2;

        if (arr[mid] == x)
            return mid;

        if (arr[mid] > x) //if-else 1 + T(n/2)
            return binarySearch(arr, l, mid - 1, x);
        return binarySearch(arr, mid + 1, r, x); //else
    }
    return -1;
}
```

- $T(n) = 1 + T(n/2)$ (si $n > 1$); $T(n) = 1$ (si $n = 1$).
- Luego: $T(n) = 1 + 1 + 1 + \cdots + T(n/2^k)$
- Al finalizar: $T(n/2^k) = T(1) \rightarrow n = 2^k$
- $\log_2 n = log_2(2^k) \rightarrow k = \log_2 n$
- $T(n) = 1 + T(n/2) = 1 + \mathcal{O}(\log n) = \mathcal{O}(\log n)$

Funciones de Recurrencia

Búsqueda Binaria

```
int binarySearch(int arr[], int l, int r, int x){
    if (r >= l) {
        int mid = l + (r - l) / 2;

        if (arr[mid] == x)
            return mid;

        if (arr[mid] > x) //if-else 1 + T(n/2)
            return binarySearch(arr, l, mid - 1, x);
        return binarySearch(arr, mid + 1, r, x); //else
    }
    return -1;
}
```

- $T(n) = 1 + T(n/2)$ (si $n > 1$); $T(n) = 1$ (si $n = 1$).
- Luego: $T(n) = 1 + 1 + 1 + \cdots + T(n/2^k)$
- Al finalizar: $T(n/2^k) = T(1) \rightarrow n = 2^k$
- $\log_2 n = log_2(2^k) \rightarrow k = \log_2 n$
- $T(n) = 1 + T(n/2) = 1 + \mathcal{O}(\log n) = \mathcal{O}(\log n)$

Funciones de Recurrencia

Búsqueda Binaria

```
int binarySearch(int arr[], int l, int r, int x){
    if (r >= l) {
        int mid = l + (r - l) / 2;

        if (arr[mid] == x)
            return mid;

        if (arr[mid] > x) //if-else 1 + T(n/2)
            return binarySearch(arr, l, mid - 1, x);
        return binarySearch(arr, mid + 1, r, x); //else
    }
    return -1;
}
```

- $T(n) = 1 + T(n/2)$ (si $n > 1$); $T(n) = 1$ (si $n = 1$).
- Luego: $T(n) = 1 + 1 + 1 + \cdots + T(n/2^k)$
- Al finalizar: $T(n/2^k) = T(1) \rightarrow n = 2^k$
- $\log_2 n = log_2(2^k) \rightarrow k = \log_2 n$
- $T(n) = 1 + T(n/2) = 1 + \mathcal{O}(\log n) = \mathcal{O}(\log n)$

Funciones de Recurrencia

MergeSort

```
void mergeSort(int arr[], int l, int r){ //l=izq, r=der (indices)
    if (l < r) {
        int m = l + (r - l) / 2; // O(1)

        mergeSort(arr, l, m); // T(n/2)
        mergeSort(arr, m + 1, r); // T(n/2)

        merge(arr, l, m, r); // O(n)
    }
}
```

- $T(n) = n + 2 \times T(n/2)$ (si $n > 1$); $T(n) = 1$ (si $n = 1$).
- $T(n) = n(1 + \log n) = n + n \log n$
- $T(n) = \mathcal{O}(n \log n)$

Funciones de Recurrencia: Ejercicios

Desarrolle las siguientes funciones de recurrencia, y determine su cota superior asintótica $\mathcal{O}(g(n))$ (tip: use árboles):

- $T(n) = T(n/2) + n^2$
- $T(n) = 4T((n/2) + 2) + n$
- $T(n) = 2T(n-1) + 1$